Presentation guide (plain-language version)

Paper: *What an SDN Controller Can Actually Observe: A Measurement and Validity Study of the InSDN OpenFlow Control Channel* Authors: Adhyan Jain, Atharva Sheersh Pandey, Kshitij Manikshete (VIT Vellore). Paper file: `research/insdn_controller/paper/main.pdf`.

How to use this guide: read Part 1 and Part 2 first (what the paper is and what is new, in simple words). Part 3 is what you say. Parts 4 to 6 tell you exactly where everything is, what goes in and what comes out. Part 7 is the frontend.

Part 1. What the paper is, in plain words

The problem. Lots of papers say: "an SDN network has a central controller that sees everything, so we can build a smart attack detector on it". But when they test their detector, they use a file of numbers (a CSV) that was computed *at the victim computer*, not *at the controller*. Nobody checks whether the controller could ever have seen those numbers.

What we did. The authors of a popular dataset (InSDN) also recorded the real messages between their controller (ONOS) and the switch. Nobody seems to have opened those recordings. We did. We read all 44 of them and asked three simple questions:

1. Can the controller actually see the flows the dataset talks about? (Answer: no.)
2. Does what the controller sees change if you change the forwarding program? (Answer: yes, massively.)
3. If we build a detector using only what the controller sees, is a high score proof it works? (Answer: no, and we show why.)

So the paper is **a careful measurement plus a warning about how to evaluate**. It is not a new detector.

Words you must know

Word	Meaning
SDN controller	The one program that tells all switches how to forward traffic
OpenFlow	The language the controller and switches use to talk
PacketIn	"I got a packet I don't know what to do with, controller, help": the switch sends it to the controller
FlowMod	The controller's answer: "install this rule in your table"
Flow entry / match fields	A rule in the switch table and what it looks at (MAC addresses only, or also IP and ports)
5-tuple flow	One connection: source IP, destination IP, source port, destination port, protocol
MAC-pair forwarding	Rules that only look at the two MAC addresses (coarse)
Per-flow forwarding	Rules for every single connection (fine)
Balanced accuracy (BAcc)	Accuracy that is fair when there are many more benign than attack samples; 0.5 = coin flip

---

Part 2. The novelty points, explained so you can say them yourself

Rule for all of them: we say "as far as our searches show", never "first". Combining known tools in a new way still counts as a contribution; we claim the combination and the measurement, not a new algorithm.

Novelty 1: "The controller never saw the flows the dataset labels"

- **In one line:** the dataset labels connections by 5-tuple, but the controller's rules never contained IP addresses or ports.
- **Analogy:** a security guard who only checks door numbers cannot tell you who walked through; a log that lists people by name was written by someone else.
- **What we did:** parsed all 44 controller recordings, read every FlowMod and flow-statistics message, counted which fields the rules look at.
- **Evidence:** 0 of 3,827 FlowMods have IP or port fields (3,811 look at MAC pairs, 16 at EtherType). The controller did receive a stream of PacketIns (0.68 to 1.29 per target packet in 9 of 12 attack scenarios).
- **Why it matters:** every InSDN paper that calls its CSV-based detector "controller-based" is using numbers the controller never had.
- **Say to the prof:** "We checked the dataset's own controller recordings. The controller never saw the 5-tuple flows the dataset labels, so CSV-trained detectors don't describe a real controller-side detector."
- **Where:** `ofgap/flowmod_check.py` -> `results/flowmod_matches.json`; `ofgap/gap_measure.py` -> `results/gap_attack_v2.json`; paper section III.

Novelty 2: "The forwarding program decides what the controller sees"

- **In one line:** same traffic, different forwarding setting, and the controller's workload and view change by hundreds of times.
- **Analogy:** a receptionist who asks every visitor for ID at every door (per-flow) is flooded with questions; one who only checks which building you belong to (MAC-pair) hardly gets asked anything.
- **What we did:** built a small Mininet + Ryu network, ran the same benign web traffic under three settings, 10 runs each, on one switch and on a three-switch chain. Then repeated the check with Ryu's own unmodified example app.
- **Evidence:** TCP PacketIns per run: 10,422 (per-flow) vs 12.3 (MAC-pair); 665 to 1,752 times more in every paired run. Three switches: 10,881 vs 18.0. A shorter idle timeout cuts the per-flow entries a poller sees from 153.1 to 36.9. Stock Ryu: 91 FlowMods, 86 MAC-pair, 5 table-miss, none with IP or port fields.
- **Honest limit:** the direction is expected from how reactive forwarding works; the value is the measured size, with confidence intervals.
- **Say to the prof:** "What a controller can observe is a property of the forwarding app, so a detector's inputs must be specified together with the forwarding setup."
- **Where:** `live/ryu_fwd.py`, `live/run_benign.py`, `live/live_table.py` -> `results/live_modes_v4_one10.json`, `live_modes_v4_linear3.json`, `stock_ryu_flowmods.json`; paper section VI.

Novelty 3: "A high score is not proof of detection" (the validity result)

- **In one line:** controller-only models reach 0.946 balanced accuracy, but a one-line rule matches it, so the score does not prove detection.
- **Analogy:** a test where all the "sick" patients were photographed in one room and all the "healthy" ones in another; a model that recognises the room gets full marks without knowing anything about illness.
- **What we did:** built detectors using only controller messages and tested them on captures they had never seen, on attack families they had never seen, and across the two target setups. Then checked what a trivial rule scores.
- **Evidence:** MLP 0.946 vs PacketIn-rate threshold 0.616 on unseen captures. But the rule "at least one IP PacketIn in the window" scores 0.995 (1 s windows) and 1.000 (5 s) inside attack sessions. InSDN records benign and attack in separate sessions, so scores cannot be separated from session recognition. XGBoost (0.952) and logistic regression (0.961) do not beat the simple MLP (0.958).
- **Say to the prof:** "We do not claim a detector. We show that on InSDN, controller-side scores can't be trusted as detection accuracy, and we say exactly what data would fix it: benign and attack traffic mixed in one session."
- **Where:** `ofgap/ctrlfeat.py`, `ofgap/exp_ctrl2.py`, `ofgap/review_checks.py` -> `results/v4_strict_W1.json`, `v4_strict_W5_xgb.json`, `v4_iprule_*.json`, `review_checks.json`; paper sections IV, V, VI.

Novelty 4: "A measurement pitfall we caught in ourselves"

- **In one line:** our first test app hid TCP traffic from the controller and made the effect look 4 times instead of about 800 times.
- **What happened:** ping and ARP rules also matched later TCP between the same hosts, so most connections never reached the controller.
- **Why it is useful to others:** it shows how an overlapping coarse rule can understate what a controller sees by two orders of magnitude.
- **Evidence:** first campaign 43.4 vs 11.0 PacketIns (4 times); fixed app 10,350 vs 11.6, later 10,422 vs 12.3 with 10 runs.
- **Where:** `live/ryu_fwd_v1.py` (the flawed app, kept on purpose) and `results/live_modes_v2.json`; paper section VI.

Novelty 5: "Everything can be rebuilt and checked" (support, not the headline)

- **In one line:** one command regenerates every table and compares it with the stored result; every capture has a hash.
- **Where:** `reproduce.sh`, `ofgap/compare_results.py`, `paper/verify_numbers.py`, `capture_hashes.sha256`, `README.md`.
- **Say it only as:** "the numbers can be checked", not as research novelty.

What is NOT new (say this before they do)

- Using control-plane data for SDN detection: LAN-SDN-NIDS (2026) and SDNFLow already do it.
- The detector: a simple model ties the neural network, and a trivial rule beats it inside sessions.
- The stock-Ryu result: it follows from that app's code.
- Related precedent: session leakage was shown on another dataset (IoT-NID); we show it for controller-side evaluation on InSDN.

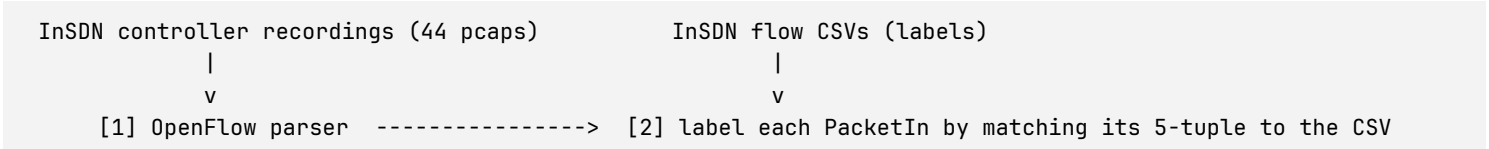
Venue honesty: workshop/conference level as is; a lower-quartile journal after revision. Not top-quartile: no new method, one dataset, no mixed benign+attack test.

Part 3. The 2-minute pitch (say this)

"Most SDN intrusion-detection papers say the controller sees everything, but they score on features computed at the victim host. InSDN's authors also recorded their controller's OpenFlow traffic, and we parsed all 44 recordings. First: no rule the controller installed ever looked at IP addresses or ports, so the connections the dataset labels were never visible to it. Second: in our own Mininet and Ryu tests, per-flow forwarding sends the controller hundreds to over a thousand times more PacketIns than MAC-pair forwarding, and Ryu's stock app also uses MAC-pair rules, so what a controller sees depends on the forwarding app. Third: models using only the controller's view reach 0.946 balanced accuracy, but a one-line rule matches that inside attack sessions, so on InSDN the number is not proof of detection. We therefore claim a measurement and a validity warning, not a new detector, and everything is reproducible from hashed captures. The missing piece, which we name, is a test with benign and attack traffic mixed in one session."

Order of honesty: (1) "0.946 is optimistic." (2) "A trivial rule scores 0.995 inside sessions." (3) "Our first campaign was flawed; we report it." (4) "Novelty is narrow; LAN-SDN-NIDS already uses control-plane telemetry."

Part 4. Architecture of the study: what goes in, what comes out



```
              |
              v
      [3] cut into 1 s / 5 s windows → 22 numbers per window (counts, distinct IPs, entropy ...)
              |
              v
      [4] add the 3 previous windows (88 numbers) → [5] models: MLP, logistic regression, XGBoost, thresholds
              |
              v
      [6] tests: unseen captures, unseen attack family, other deployment, session probes, trivial rule
              |
              v
      results/*.json → tables in the paper (make_tables.py) → numbers checked (verify_numbers.py)

 Separate live experiment (no InSDN data):  Ryu app (3 modes) + Mininet → control-channel pcap → live_table.p
```

#	Step	Input	Output	File
0	Dataset	InSDN archives (hashed, see `data/MANIFEST.md`)	44 controller pcaps (11 benign, 33 attack)	`data/raw/insdn_pcaps/ex/`
1	Parse OpenFlow	one pcap	a list of messages: PacketIn, PacketOut, FlowMod, FlowRemoved, stats replies (7,381,679 in total; 12,217 bytes lost)	`ofgap/ofparse.py`
2	Label	each PacketIn's embedded packet + CSV Flow IDs	tag per PacketIn: attack-in-CSV, attack-IP-only, normal, unmatched IP, non-IP	`ofgap/ctrlfeat.py`, `attacker_ips.py`
3	Windows and features	message stream, window length	22 numbers per window	`ofgap/ctrlfeat.py` -> `feats_v4/` (regenerable)
4	History	window rows	88 inputs (current + 3 previous), causal	`ofgap/exp_ctrl.py`
5	Models	88 inputs + window label (attack or not)	one prediction per window; recall, FPR, balanced accuracy	`ofgap/exp_ctrl2.py` (GPU)
6	Tests	predictions	10-seed mean with 95% interval; bootstrap; trivial-rule scores	`exp_ctrl2.py`, `review_checks.py`
7	Granularity check	the same pcaps	which fields FlowMods carry	`flowmod_check.py`, `gap_measure.py`
8	Live study	Ryu app + Mininet, benign traffic	control-channel pcaps and client logs	`live/run_benign.py` (root)
9	Live analysis	those pcaps	PacketIns, FlowMods, entries per poll, per mode	`live/live_table.py`
10	Baselines	InSDN CSV	host-side XGBoost score (contrast)	`ofgap/hostside_baseline.py`

#	Step	Input	Output	File
11	Paper	result JSONs	LaTeX tables, PDF	`ofgap/make_tables.py`, `paper/`

Model input and output in one sentence: input = 88 numbers describing a few seconds of control-channel messages; output = "attack or not" for that window. The CSV flow features are never an input; the CSV only supplies labels.

Training device: all models train on the RTX 4060 GPU.

Part 5. Where is everything

What you need to show	Where
The paper (PDF and LaTeX)	`research/insdn_controller/paper/main.pdf`, `main.tex`
Every number's source	`research/insdn_controller/results/*.json`
Re-run everything	`research/insdn_controller/reproduce.sh` (README next to it explains)
Number check against the paper	`python research/insdn_controller/paper/verify_numbers.py`
Capture hashes	`research/insdn_controller/capture_hashes.sha256`
Dataset sources and hashes	`data/MANIFEST.md`
Raw captures (not in git)	`data/raw/insdn_pcaps/` and `research/insdn_controller/live/`
The older Proteus system (GAN, drift)	`backend/`, `api/`, `frontend/`; not part of this paper's claims

---

Part 6. Questions you will get

Question	Short answer
Does it detect attacks?	Not proven. It separates InSDN's attack captures from benign ones; a trivial rule matches it inside sessions.
Why not use the 99% CSV numbers?	The controller never receives those features; that gap is the paper.
Logistic regression ties your neural net: what is the model contribution?	None. The contribution is the measurement, the validity analysis and the reproducible pipeline.
Is 10 seeds enough?	They vary folds and initialisation on the same 44 captures; they are not new testbeds. The bootstrap over captures is wider and is in the paper.
Isn't this obvious for ONOS?	The direction is; the benchmark is still used as if the controller saw those flows, and nobody had measured the size.
Why no oversampling for rare classes?	Some classes have only 6 to 12 attack windows because the CSV labels cover part of each capture; augmenting would copy those windows and risks leakage. It is listed as future work.

Question	Short answer
What would make it stronger?	Benign and attack traffic mixed in one session, the gap on another controller (OpenDaylight), and a re-implemented published detector.
What did you not do?	No attack traffic generated by us; no published-detector re-implementation; one dataset; Ryu apps only as second controller.

Numbers to remember: 44 captures; 3,827 FlowMods, 0 with IP/ports; 0.946 vs 0.616; trivial rule 0.995; 10,422 vs 12.3 (665 to 1,752 times); 153.1 to 36.9; 3-switch 10,881 vs 18.0; stock Ryu 91 / 86 / 5; host-side 0.9998.

Part 7. The frontend: architecture and what each page is for

```
Browser  →  Next.js console (port 3000)  →  FastAPI (port 8000)  →  files on disk
              10 pages                        read-only endpoints      results/results.pkl            (demo
                                                                       models/releases/vNNN           (demo
                                                                       research/insdn_controller/results/*.j
```

Start it: `.venv/bin/uvicorn api.server:app --port 8000` and `cd frontend && npm run dev` (page: http://localhost:3000). Every panel has a "What this shows" line; use it as your talking point.

Two groups of pages. Do not mix them up in front of the prof.

Group	Pages	Data source	Status
This paper	**Controller Telemetry**, **Topology**	`research/insdn_controller/results/*.json` through `/api/controller-telemetry`	Real results from the paper
The older Proteus demo	Overview, Live Detection, Experiments, Drift Monitor, Synthetic Lab, Fidelity Gate, Adaptation, Audit Log, Models	`results/results.pkl` (a small single-seed demo run)	Demo scale; its data source is `synthetic-fallback`; **not evidence for the paper**

Page	What it shows	API endpoint	What to say
Controller Telemetry	Detection scores, forwarding-mode tables (1 and 3 switches), stock-Ryu result, host-side contrast	`/api/controller-telemetry`	"This is the paper's evidence, read straight from the result files."
Topology	The two Mininet networks (1 switch, 3-switch chain) and recorded PacketIn counts	`/api/controller-telemetry`	"This is the testbed behind Novelty 2."
Overview	The 8-stage closed-loop pipeline (traffic, classify, drift,	several `/api/...`	"This is the earlier Proteus system architecture, a separate project."

Page	What it shows	API endpoint	What to say
	generate, gate, retrain, redeploy)		
Models	Demo release, datasets (InSDN 343,889 rows; CICIDS2017 2.1 M rows)	`/api/models`, `/api/datasets/summary`	"Dataset sizes and class mix; shows why rare classes are tiny."
Experiments, Drift, Synthetic Lab, Fidelity Gate, Adaptation, Live Detection, Audit Log	The demo's drift detection, GAN training, fidelity check and retraining	`/api/experiments/*`, `/api/drift/*`, `/api/gan/*`, `/api/gate/*`, `/api/adaptation/*`	"These belong to the older Proteus demo; its gain is not evidence that synthetic data helps."

Frontend architecture of the study itself: Topology page = the testbed; Controller Telemetry page = what was measured; the paper sections map one-to-one to those two pages. The 8-stage pipeline diagram on Overview is the Proteus system architecture; it does not describe the paper's pipeline (that one is Part 4).

Part 8. If something goes wrong

- A number is questioned: open `results/<file>.json` named in Part 2; every number comes from one.
- You do not know: say "I measured X but not Y; Y is future work." Never improvise a number.
- They ask about the older GAN system: say it is a separate project whose research findings are in the repo README and that this paper makes no GAN claim.